

Lab 4

Simply Simulation

Introduction

This lab consists of three parts:

1. Write a short program in a simple 8-bit assembly language.
2. Assemble the simple program into binary format by hand (the good early days!).
3. Write a simulator for our 8-bit assembly language.

The Simply ISA

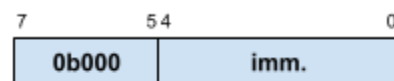
“Simply” is a minimal instruction set architecture ([ISA](#)). It is designed according to [RISC principles](#) (simplicity, performance, [load/store architecture](#), and orthogonal encoding). It is an 8-bit architecture: registers are 8-bit wide, instructions are 8-bit wide, memory is byte addressable (i.e., address 0 corresponds to the first byte, address 1 corresponds to the second byte, etc.) and holds $2^8 = 256$ bytes. The architectural state consists of an 8-bit program counter (PC), four 8-bit general-purpose registers (X0, X1, X2, and X3) and 256 bytes of memory.

The Simply ISA consists of the following eight instructions:

1. **Instruction:** *BRA imm.*

Description: Fetch the next instruction at the *next PC + immediate* (sign-extended to 8 bits)

Encoding:



2. **Instruction:** *BZ X1 = 0 imm.*

Description: Fetch the next instruction at the *next PC* unless the content of X1 is zero, in which case fetch the next instruction at the *next PC + immediate* (sign-extended to 8 bits)

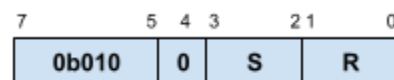
Encoding:



3. **Instruction:** *LD R, [S]*

Description: Load register *R* with the content of memory pointed to by register *S*

Encoding:



4. **Instruction:** *LDI X0, imm.*

Description: Load X0 with the immediate value, sign-extended to 8 bits

Encoding:

5. **Instruction:** *ST R, [S]*

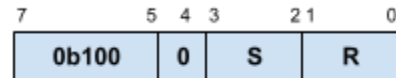
Description: Store the content of register *R* at the address in memory pointed to by register *S*



Encoding:

6. **Instruction:** *ADD R, S*

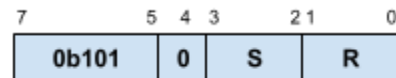
Description: Add the content of register *R* and *S*, store the result in register *R*



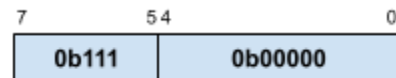
Encoding:

7. **Instruction:** *HLT*

Description: Stop the machine



Encoding:



Register encoding scheme

$X0 = 0b00$, $X1 = 0b01$, $X2 = 0b10$, $X3 = 0b11$.

Initial state

All registers (*PC*, *X0*, *X1*, *X2*, and *X3*) initially contain 0x00. The program is loaded at address 0 of memory.

Square

As a first step, you must write a program in the Simply assembly language that takes the value stored at memory location 1, squares its value, and stores the resulting value in location 2. Write this program in a file called **square.s**. The format of such a file is as follows:

- The file consists of *labels*, *instructions*, *literals*, and *comments*.
- A *label* is a string starting with an alphabetic character or an underscore and followed by an arbitrary number alphabetic characters, underscores, or digits, and terminated with a colon; a label must start in the first column of the text.
- *Instructions* and *literals*, one per line, must be preceded by a single tab character.
- An instruction's immediate value must be specified using either a literal or a label.
- *Comments* start with the '#' character and extend to the rest of the line; they may start anywhere on that line.

Sample assembly program:

```
# A simple example.
start:
    BRA L1          # This is a branch instruction
L1:    LDI X0, IN
        HLT
IN:    42
```

Binary

The second step of this lab consists of assembling the program written in the previous step into a binary. You can write a small program to help you with that step or use an editor capable of editing binary files.

Assembling the previous program into a binary file called **simple.out** and running the command **od -t x1 simple.out** would produce the following output:

```
00000000 00 63 e0 2a
00000004
```

Simply simulator

In the third part of this lab you must write a simulator that can take the binary resulting from the second step, simulating the execution of the program on an hypothetical CPU implementing the Simply ISA.

The simulator, called **simfun**, should simulate the state of the simulated machine, starting with its initialization, read in the binary of the program whose execution we are simulating, and stop when encountering the HLT instruction. Before exiting, the simulator should output all the architectural state that has changed in the course of the simulation. Compute the changes from after loading the binary into memory.

The output of your program should look similar, but not necessarily identical, to a sample run reproduced below. The key is the last line. It shows that the memory address content reserved for square's output has changed from 0 to 25 (assuming that the memory location reserved for the input was initially set to 5).

```
./simfun < asm.out
HLT encountered at PC = 20
PC: 0 --> 20
X0: 0 --> 2
```

```
X1: 0 --> 25  
X2: 0 --> 5  
X3: 0 --> 1  
mem[2]: 0 --> 25
```

What to hand in

Hand in **square.s**, **square.out**, and **simfun.c**. Don't forget to add your name somewhere in all your text files.